A Public Stagewise End-to-End Full-Key Recovery Attack on the `main.c` Implementation of SEPAR

2026-04-15

Abstract

We study the concrete SEPAR implementation in `SEPAR/main.c`, not an abstract idealized design. In the chosen-IV, chosen-plaintext, chosen-ciphertext, reset model, we give a public end-to-end full-key recovery attack on the 256-bit implementation key. The attack has a stochastic front-end and a deterministic exact core. The stochastic component is a chosen-IV search for suitable stage-8 and inward contexts. Once those IV sets are hit, every subsequent step is exact: an exact public stage-8 bootstrap for K_8; an exact stagewise inward search in which already-solved outer stages are peeled on fresh IVs by exact fixed-pair state recovery; and an exact stage-1 closure from the public stage-2 reduced table. Final acceptance is exact: a 256-bit key is accepted if and only if it reproduces the full 2^{16}-entry one-word oracle table on the successful IV. We define the exact attack primitives used by the implementation, prove conditional exactness, report complete recovery on difficult random keys, and give a full-program Monte Carlo over 100 pseudorandom 256-bit keys. Under the explicit Monte Carlo outer-search profile $(N_{\text{out}}, B, \texttt{search-seed}) = (512, 32, 1)$, the observed random-key success rates were 9/100 at inward budget $N = 2$, 45/100 at $N = 4$, 89/100 at $N = 8$, and 100/100 at every tested budget $N \geq 16$.

1 Introduction

The target is the concrete word-oriented implementation in `SEPAR/main.c`. The relevant attack surface is the resettable 16-bit one-word oracle induced by that code. For every fixed IV, the implementation exposes a public permutation

$$F_{\text{IV}} : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$$

on one-word inputs. Because the block size is only 16 bits, the entire codebook is observable with 2^{16} chosen encryptions and can then be manipulated exactly.

The attack model is:

- chosen IV;
- chosen plaintext and chosen ciphertext;
- exact state reset between oracle calls.

Under that model, the implementation gives exact public access to

$$F_{\text{IV}}(x) = \text{Enc}_{K,\text{IV}}(x), \qquad F_{\text{IV}}^{-1}(y) = \text{Dec}_{K,\text{IV}}(y),$$

and the implementation used in the attack constructs the inverse locally by inverting the public encryption table.

Main claim. The public attack implemented in

`separ_public_stagewise_attack.c`

recovers the full 256-bit key by:

1. a chosen-IV beam search and exact hybrid public bootstrap for K_8;
2. an exact stagewise inward search for $K_7, K_6, \ldots, K_2$ in which solved outer stages are peeled on fresh IVs by exact fixed-pair state recovery;
3. an exact stage-1 closure from the public stage-2 reduced table;
4. exact full-codebook verification of the recovered 256-bit key.

What is proved below is *conditional exactness*: once the sampled IV sets contain the true exact candidates required by the attack, the remainder of the recovery is deterministic and exact.

2 Exact implementation model

2.1 Stage-local permutation

Let the eight 32-bit stage keys be

$$K_i = (k_{i,0}, k_{i,1}) \in \mathbb{Z}_{2^{16}} \times \mathbb{Z}_{2^{16}}, \qquad 1 \le i \le 8.$$

Let $\Sigma : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$ be the nibblewise S-box layer induced by the fixed tables S_1, S_2, S_3, S_4 in `main.c`, and Σ^{-1} the inverse layer induced by `IS1`, ..., `IS4`.

Let $L : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$ be the exact linear layer implemented by `sep_rotl16`. If

$$x = (a, b, c, d) \in (\mathbb{Z}_{16})^4$$

in nibble notation, define

$$M(a, b, c, d) = (a \oplus c,\ b \oplus d,\ c \oplus b,\ d \oplus a),$$

identify nibble tuples with 16-bit words, and set

$$L(x) = M(x) \oplus \mathrm{rotl}_{16}(M(x), 12) \oplus \mathrm{rotl}_{16}(M(x), 8).$$

Its inverse is exactly `sep_inrotl16`.

For each stage i, the code derives two further 16-bit subkeys $\kappa_{i,2}, \kappa_{i,3}$ from $(k_{i,0}, k_{i,1})$:

$$\begin{aligned} u_i &= \mathrm{rotl}_{16}(k_{i,0}, 6), & v_i &= \mathrm{rotl}_{16}(k_{i,1}, 10), \\ \kappa_{i,2} &= \big(u_i \vee (S_1((u_i \gg 6) \bmod 16) \ll 6)\big) \oplus (i+2), \\ \kappa_{i,3} &= \big(v_i \vee (S_1((v_i \gg 6) \bmod 16) \ll 6)\big) \oplus (i+3). \end{aligned}$$

With $A_t(x) = x \oplus t$, the stage-local encryption permutation implemented by `enc_block` is exactly

$$E_i = A_{\kappa_{i,2} \oplus \kappa_{i,3}} \circ \Sigma \circ A_{k_{i,0} \oplus k_{i,1}} \circ L \circ \Sigma \circ A_{\kappa_{i,3}} \circ L \circ \Sigma \circ A_{\kappa_{i,2}} \circ L \circ \Sigma \circ A_{k_{i,1}} \circ L \circ \Sigma \circ A_{k_{i,0}}, \quad (1)$$

and $D_i = E_i^{-1}$ is exactly `dec_block`.

2.2 Translated stage action and one-word encryption

For a state word $s_i \in \mathbb{Z}_{2^{16}}$, define the translated stage permutation

$$B_i^{(s_i)}(x) = E_i(x + s_i), \qquad \big(B_i^{(s_i)}\big)^{-1}(y) = D_i(y) - s_i,$$

where all additions and subtractions are in $\mathbb{Z}_{2^{16}}$.

For a reset IV with state $(s_1, \ldots, s_8)$, the public one-word encryption permutation is

$$F_{\mathrm{IV}} = B_8^{(s_8)} \circ B_7^{(s_7)} \circ \cdots \circ B_1^{(s_1)}.$$

2.3 State update law

Write the eight state words as $(s_1, \ldots, s_8)$ and the intermediate stage outputs as

$$v_{12}, v_{23}, v_{34}, v_{45}, v_{56}, v_{67}, v_{78}, c.$$

From `Separ_Encryption` in `main.c`, the exact state update after one word is

$$s_2' = s_2 + v_{12} + v_{56} + s_6, \tag{2}$$
$$s_3' = s_3 + v_{23} + v_{34} + s_4 + s_1, \tag{3}$$
$$s_4' = s_4 + v_{12} + v_{45} + s_8, \tag{4}$$
$$s_5' = s_5 + v_{23} + L', \tag{5}$$
$$s_6' = s_6 + v_{12} + v_{45} + s_7, \tag{6}$$
$$s_7' = s_7 + v_{23} + v_{67}, \tag{7}$$
$$s_8' = s_8 + v_{45}, \tag{8}$$
$$s_1' = s_1 + s_5 + v_{34} + 2v_{23} + v_{78}. \tag{9}$$

The attack below uses only reset one-word tables, so these equations matter only because they determine which state word appears as the output translation at each stage.

3 Exact public tables used by the attack

3.1 Recursive current tables

For $m \in \{1, \ldots, 7\}$ define the inner cascade

$$R_m := B_m^{(s_m)} \circ B_{m-1}^{(s_{m-1})} \circ \cdots \circ B_1^{(s_1)}.$$

Definition 1 (Current table at level m)**.** *Assume $K_{m+1}, \ldots, K_8$ and $s_{m+2}, \ldots, s_8$ are known for a fixed IV. The current table at level m is the public table*

$$C_m : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$$

computed recursively by

$$C_7 = D_8 \circ F_{\mathrm{IV}}, \qquad C_m = D_{m+1}\big(C_{m+1} - s_{m+2}\big) \quad (1 \le m \le 6).$$

Lemma 1 (Exact translation form)**.** *For every $m \in \{1, \ldots, 7\}$,*

$$C_m(x) = R_m(x) + s_{m+1} \qquad \textit{for all } x \in \mathbb{Z}_{2^{16}}.$$

Proof. For $m = 7$,

$$F_{\mathrm{IV}}(x) = E_8(R_7(x) + s_8),$$

hence

$$C_7(x) = D_8(F_{\mathrm{IV}}(x)) = R_7(x) + s_8.$$

Assume the statement holds at level $m + 1$. Then

$$C_{m+1}(x) = R_{m+1}(x) + s_{m+2} = E_{m+1}(R_m(x) + s_{m+1}) + s_{m+2},$$

so

$$C_m(x) = D_{m+1}(C_{m+1}(x) - s_{m+2}) = R_m(x) + s_{m+1}.$$

□

Thus the exact public object attacked at level m is the inner cascade R_m observed through a single output translation by s_{m+1}.

3.2 Exact low-byte score

Let

$$H(y) = \left\lfloor \frac{y}{2^8} \right\rfloor \in \mathbb{Z}_{256}$$

be the output high byte. For a public table $T : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$, a candidate low byte $b \in \mathbb{Z}_{256}$, and a row index $r \in \mathbb{Z}_{256}$, define

$$\sigma_{T,b}(r) := \left|\left\{H\big(T(256r + \lambda) - b\big) : \lambda \in \mathbb{Z}_{256}\right\}\right|,$$

and the exact total score

$$\Sigma(T; b) := \sum_{r=0}^{255} \sigma_{T,b}(r).$$

This is exactly the quantity computed by `exact_low_byte_scan_u16()`.

3.3 Projected quotient cycles, representatives, and verifier

Define the 4-bit projection

$$\mathrm{proj}(y) := H(y) \bmod 16 \in \mathbb{Z}_{16}.$$

For a stage key pair K_i, define the stage-i quotient map

$$q_{i,K_i}(h) = \mathrm{proj}(E_i(256h)), \qquad h \in \mathbb{Z}_{16},$$

and the full high-byte quotient map

$$Q_{i,K_i}(h) = H(E_i(256h)), \qquad h \in \mathbb{Z}_{256}.$$

The implementation uses the exact auxiliary sets

$$\Delta_0 = \{1, 2, 4, 8, 15, 16\},$$

$$\Omega = \{0, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 200, 233, 255\},$$

$$\mathcal{R} = \{0x00, 0x55, 0xAA, 0xFF\}.$$

The corrected implementation uses the same universal invisible-bit set for every stage $i \in \{2, \ldots, 8\}$:

$$P_i = \{6, 7, 12, 13, 14, 15, 18, 19, 20, 21, 22, 23\}.$$

Hence

$$|P_i| = 12, \qquad |V_i| = 20,$$

and every representative table enumerates exactly

$$2^{20} = 1{,}048{,}576$$

visible masks. Stages $2, \ldots, 7$ canonicalize 16-cycles by rotation to first entry 0; stage 8 uses raw cycles.

For any public table T, define the exact projected edge-count matrix

$$M_T(a,b) := \left|\{(x,\delta) \in \mathbb{Z}_{2^{16}} \times \Delta_0 : \mathrm{proj}(T(x)) = a,\ \mathrm{proj}(T(x+\delta)) = b\}\right|, \qquad a,b \in \mathbb{Z}_{16}.$$

If $c = (c_0, \ldots, c_{15})$ is a 16-cycle, define the exact cycle score

$$S_T(c) = \sum_{j=0}^{15} M_T(c_j, c_{j+1 \bmod 16}).$$

To lift a projected cycle to a 256-point high-byte order, the implementation builds the exact 8-bit transition matrix

$$N_T(u,v) := \left|\{(x,\delta) \in \mathbb{Z}_{2^{16}} \times \Delta_0 : H(T(x)) = u,\ H(T(x+\delta)) = v\}\right|, \qquad u,v \in \mathbb{Z}_{256},$$

solves exact 16×16 assignments between adjacent cycle layers by the Hungarian algorithm, and accepts only wrap permutations forming one 16-cycle.

Given a recovered cycle/order pair, the implementation enumerates every compatible representative-group base key and expands all invisible masks. It then checks exact equality against the reconstructed order targets first on Ω and then on all 256 values. Finally it exact-scores the surviving candidates by

$$V_i(K_i; T) := \sum_{r \in \mathcal{R}} \min_{b \in \mathbb{Z}_{256}} \left|\left\{H\big(D_i(T(256r+\lambda)) - b\big) : \lambda \in \mathbb{Z}_{256}\right\}\right|.$$

This is the verifier used both in the outer bootstrap and in the stagewise inward search.

4 The outer public bootstrap for K_8

4.1 Weak-IV beam search

For an IV, define the exact row score

$$\rho(\mathrm{IV}) := \left|\{H(F_{\mathrm{IV}}(\lambda)) : \lambda \in \mathbb{Z}_{256}\}\right|.$$

The implementation keeps the B IVs with smallest $\rho(\mathrm{IV})$. This is a search-ordering device only; it is not part of the acceptance criterion.

4.2 Hybrid exact stage-8 recovery

For each IV in the beam, the implementation:

1. computes the exact stage-8 quotient/bootstrap candidate set by projected-cycle maximization and exact verifier scoring;
2. closes that set under the independent MSB toggles
$$(a,b) \mapsto (a \oplus \epsilon_0 2^{15},\ b \oplus \epsilon_1 2^{15});$$
3. exact-scores the resulting small pool by the additive differential score
$$A_{\mathrm{IV}}(K_8) := \sum_{\delta \in \Delta_0} \max_{\Delta \in \mathbb{Z}_{2^{16}}} \left|\{x : D_8(F_{\mathrm{IV}}(x+\delta); K_8) - D_8(F_{\mathrm{IV}}(x); K_8) = \Delta\}\right|;$$
4. aggregates those exact scores across the beam and outputs the best K_8.

5 Exact stagewise inward search from K_7 to K_2

5.1 Exact fixed-pair peeling on fresh IVs

The inward phase is not a one-IV cascade. Once the outer bootstrap has fixed K_8, the program samples a fresh IV set independently at each target stage. On each such IV it peels every already-solved outer stage by exact state recovery for the fixed known stage key, and only then attacks the next unknown stage.

Fix a target stage $t \in \{7, 6, 5, 4, 3, 2\}$ and suppose a branch has already fixed

$$K_{t+1}, K_{t+2}, \ldots, K_8.$$

Let η be a fresh IV. Define

$$C_7^{(\eta)} := D_8 \circ F_\eta.$$

If K_8 is correct, then by Lemma 1,

$$C_7^{(\eta)}(x) = R_7^{(\eta)}(x) + s_8^{(\eta)}.$$

For a fixed stage $m \in \{7, \ldots, t+1\}$ and a public table T, define the minimizing low-byte set

$$L_m(T) := \arg\min_{b \in \mathbb{Z}_{256}} \Sigma(T; b).$$

For a fixed known pair K_m, define the exact fixed-pair state solver

$$\mathcal{S}_m(T; K_m) := \arg \min_{\substack{s=b+256h \\ b \in L_m(T),\, h \in \mathbb{Z}_{256}}} V_m(K_m; T - s), \tag{10}$$

with lexicographic tie-break on s. This is exactly `recover_known_pair_state_fast()`.

Starting from $C_7^{(\eta)}$, the program peels already-solved outer stages by

$$\widehat{s}_{m+1}^{(\eta)} := \mathcal{S}_m(C_m^{(\eta)}; K_m), \tag{11}$$

$$C_{m-1}^{(\eta)} := D_m\left(C_m^{(\eta)} - \widehat{s}_{m+1}^{(\eta)}; K_m\right), \qquad m = 7, 6, \ldots, t+1. \tag{12}$$

If every known outer pair on the current branch is correct and every fixed-pair state recovery returns the true state word, then

$$C_t^{(\eta)}(x) = R_t^{(\eta)}(x) + s_{t+1}^{(\eta)}.$$

5.2 Exact target-stage candidate sets

For a fixed target stage t and fresh IV η, let

$$\mathcal{A}_t(\eta; K_{t+1}, \ldots, K_8)$$

denote the set of all minimum-verifier rows produced by applying the exact stage-t scan of Sections 3.2–3.4 to the prepared table $C_t^{(\eta)}$. Every element of $\mathcal{A}_t$ is a pair

$$(K_t, s_{t+1}) \in (\mathbb{Z}_{2^{16}})^2.$$

Write

$$\Pi_{\text{pair}} \mathcal{A}_t(\eta; K_{t+1}, \ldots, K_8)$$

for the set of stage-key pairs appearing in those rows.

Let

$$\Omega_t(N) = \{\eta_{t,1}, \ldots, \eta_{t,N}\}$$

be the first N fresh IVs in the deterministic stage-t IV stream. Define the exact stagewise candidate-pair set

$$\mathcal{P}_t^{(N)}(K_{t+1}, \ldots, K_8) := \bigcup_{\eta \in \Omega_t(N)} \Pi_{\text{pair}} \mathcal{A}_t(\eta; K_{t+1}, \ldots, K_8). \tag{13}$$

The implementation also records the exact hit multiplicity

$$h_t^{(N)}(K_t) := \left|\{\eta \in \Omega_t(N) : K_t \in \Pi_{\text{pair}} \mathcal{A}_t(\eta; K_{t+1}, \ldots, K_8)\}\right|, \tag{14}$$

sorts candidates by decreasing $h_t^{(N)}$, and recursively explores all distinct pairs in that exact order. This ordering affects runtime only: no candidate in (13) is discarded.

5.3 Exact recursive search tree

Define recursively the exact search procedures

$$\mathfrak{S}_t^{(N)}(K_{t+1}, \ldots, K_8), \qquad t = 7, 6, \ldots, 2,$$

as follows.

For $t \in \{7, 6, 5, 4, 3\}$:

1. build $\mathcal{P}_t^{(N)}(K_{t+1}, \ldots, K_8)$ by (13);
2. enumerate all $K_t \in \mathcal{P}_t^{(N)}$ in decreasing order of (14);
3. recurse to
$$\mathfrak{S}_{t-1}^{(N)}(K_t, K_{t+1}, \ldots, K_8).$$

For $t = 2$, there is no further pair recursion. Instead, for every $\eta \in \Omega_2(N)$ the program:

1. prepares the exact public stage-2 current table $C_2^{(\eta)}$ by fixed-pair peeling of the already-solved suffix;
2. computes the exact minimum-verifier row set $\mathcal{A}_2(\eta; K_3, \ldots, K_8)$;
3. for every row $(K_2, s_3) \in \mathcal{A}_2(\eta; K_3, \ldots, K_8)$, invokes the exact stage-1 closure of Section 6;
4. accepts only if the recovered 256-bit key reproduces the entire 2^{16}-entry one-word oracle table on the same IV.

Why stage 1 is separate. The projected-family scan exists only for stages $2, \ldots, 8$. Stage 1 has no visible successor stage and therefore no stage-1 quotient-cycle analogue. It is closed exactly from the stage-2 reduced table.

Proposition 1 (Monotonicity in the inward budget)**.** *Fix a key, the outerly recovered K_8, and the deterministic per-stage IV streams. If the stagewise attack succeeds with inward budget N, then it also succeeds with every larger budget $N' \geq N$.*

Proof. For every stage t, the sampled IV set $\Omega_t(N)$ is a prefix of $\Omega_t(N')$. Hence

$$\mathcal{P}_t^{(N)}(K_{t+1}, \ldots, K_8) \subseteq \mathcal{P}_t^{(N')}(K_{t+1}, \ldots, K_8)$$

for every fixed suffix. Thus every branch explored at budget N is also explored at budget N'. At stage 2 the tested IV set is likewise prefix-monotone, so any successful stage-2 IV at budget N is still tested at budget N'. Therefore success is monotone. □

6 Exact stage-1 closure from the public stage-2 table

6.1 Reduction to a single translated stage

Assume the stagewise recursion has recovered the true (K_2, s_3) on some successful stage-2 IV. Then

$$C_2(x) = E_2(E_1(x + s_1) + s_2) + s_3.$$

Define

$$W_2(x) = D_2(C_2(x) - s_3; K_2).$$

Then exactly

$$W_2(x) = E_1(x + s_1) + s_2. \tag{15}$$

6.2 Low-byte recovery for s_2

The implementation first applies the same exact low-byte scan to W_2. For each candidate

$$s_2^\star \in \mathbb{Z}_{2^{16}},$$

it forms

$$C^\star(x) = W_2(x) - s_2^\star.$$

Any true stage-1 solution must satisfy

$$C^\star(x) = E_1(x + s_1; K_1).$$

6.3 Step 1: exact row-signature inversion

For $u \in \mathbb{Z}_{256}$ define the 16-point row signature

$$R_u^{C^\star}(h) = \mathrm{proj}(C^\star(u + 256h)), \qquad h \in \mathbb{Z}_{16}.$$

Write

$$s_1 = s_{1,\mathrm{lo}} + 256\, s_{1,\mathrm{mid}} + 4096\, s_{1,\mathrm{hi}},$$

with $s_{1,\mathrm{lo}} \in \mathbb{Z}_{256}$ and $s_{1,\mathrm{mid}}, s_{1,\mathrm{hi}} \in \mathbb{Z}_{16}$.

For a candidate $s_{1,\mathrm{lo}}$, the routine `stage1_step1_candidates_u16()` checks the exact carry partition:

> all rows with $u + s_{1,\mathrm{lo}} < 256$ must share one common signature σ, and all rows with $u + s_{1,\mathrm{lo}} \geq 256$ must share the one-step rotation $\mathrm{rot}_1(\sigma)$.

For each surviving $s_{1,\mathrm{lo}}$ and each $s_{1,\mathrm{mid}} \in \mathbb{Z}_{16}$, the implementation rotates σ by $16 - s_{1,\mathrm{mid}}$ and inverts it through the exact precomputed family

$$\nu_{a,b,c,d} : \mathbb{Z}_{16} \to \mathbb{Z}_{16},$$

where

$$\nu_{a,b,c,d}(x) = S_2\big(t_4(x) \oplus a \oplus b\big) \oplus c \oplus d.$$

with

$$\begin{aligned} t_1(x) &= S_2(x \oplus a), \\ t_2(x) &= S_2(t_1(x) \oplus b), \\ t_3(x) &= S_2(t_2(x) \oplus c), \\ t_4(x) &= S_2(t_3(x) \oplus d). \end{aligned}$$

This yields exact candidates for

$$s_{1,\mathrm{lo}}, \quad s_{1,\mathrm{mid}}, \quad a, \quad b, \quad c, \quad d.$$

6.4 Step 2: exact lift to full K_1 candidates

The routine next uses exact inverse-image tables for the derived-key high nibbles of $\kappa_{1,2}$ and $\kappa_{1,3}$. For each surviving $(s_{1,\mathrm{lo}}, s_{1,\mathrm{mid}}, a, b, c, d)$, each

$$\ell_6 \in I_2(c), \qquad \beta \in I_3(d),$$

and each 12-bit free mask $f \in \mathbb{Z}_{2^{12}}$, it constructs

$$k_{1,0} = \ell_6 \ \vee \ \big((f \bmod 4) \ll 6\big) \ \vee \ (a \ll 8) \ \vee \ \left(\left(\left\lfloor \frac{f}{4} \right\rfloor \bmod 16\right) \ll 12\right), \tag{16}$$

$$k_{1,1} = (\beta \bmod 4) \ \vee \ \left(\left(\left\lfloor \frac{f}{64} \right\rfloor \bmod 64\right) \ll 2\right) \ \vee \ (b \ll 8) \ \vee \ \left(\left(\left\lfloor \frac{\beta}{4} \right\rfloor \bmod 16\right) \ll 12\right). \tag{17}$$

The resulting pair is exact-tested on the 12-bit slice

$$X_{12} = \{0, \ldots, 4095\}$$

by comparing nibble (7..4) values:

$$\left\lfloor \frac{E_1((x + s_{1,\mathrm{lo}} + 256 s_{1,\mathrm{mid}}) \bmod 2^{12}; K_1)}{2^4} \right\rfloor \bmod 16 = \left\lfloor \frac{C^\star(x)}{2^4} \right\rfloor \bmod 16 \qquad \text{for all } x \in X_{12}.$$

6.5 Step 3: exact recovery of the top nibble of s_1

For each surviving pair K_1, the implementation scans all 16 possibilities for $s_{1,\mathrm{hi}}$ and performs full exact verification:

$$E_1(x + s_1; K_1) = C^\star(x) \qquad \text{for all } x \in \mathbb{Z}_{2^{16}}.$$

If exactly one triple (K_1, s_1, s_2) survives, the stage-1 closure succeeds.

Theorem 1 (Exactness of the stage-1 closure)**.** *Let W_2 be the exact public table of* (15)*. If there is a unique triple (K_1, s_1, s_2) satisfying*

$$W_2(x) = E_1(x + s_1; K_1) + s_2 \qquad \textit{for all } x \in \mathbb{Z}_{2^{16}},$$

then `stage1_finish_from_stage2_current()` *returns exactly that triple.*

Proof. The routine is exhaustive over all candidates for the low and high bytes of s_2, all exact row-signature solutions for $(s_{1,\mathrm{lo}}, s_{1,\mathrm{mid}}, a, b, c, d)$, all inverse-image candidates from $I_2(c)$ and $I_3(d)$, all 2^{12} free masks in (16)–(17), and all 16 possibilities for $s_{1,\mathrm{hi}}$. Every surviving candidate is tested by exact equality either on the 12-bit slice or on the full 2^{16}-entry table, and the routine accepts only if the number of full solutions is exactly one. Hence a unique solution is returned exactly. □

7 Public full-key acceptance and conditional correctness

Definition 2 (Exact public acceptance test)**.** *Given recovered pairs*

$$\widehat{K} = (\widehat{K}_1, \ldots, \widehat{K}_8),$$

rebuild the candidate one-word table

$$\widehat{F}_{\mathrm{IV}}$$

under the same IV and accept if and only if

$$\widehat{F}_{\mathrm{IV}} = F_{\mathrm{IV}}$$

as functions on $\mathbb{Z}_{2^{16}}$.

Because the block size is only 16 bits, this is an exact comparison of two 65536-entry tables.

Theorem 2 (Conditional exactness of the full public attack)**.** *Fix the exact beam IV set used by the outer stage-8 search and the exact fresh-IV sets $\Omega_t(N)$ used by the stagewise search. Assume:*

(a) the outer hybrid bootstrap ranks the true K_8 first;

(b) for each $t \in \{7, 6, 5, 4, 3\}$, on the branch

$$(K_{t+1}, \ldots, K_8) = (K^\star_{t+1}, \ldots, K^\star_8),$$

the true pair $K^\star_t$ belongs to the exact candidate set

$$\mathcal{P}^{(N)}_t(K^\star_{t+1}, \ldots, K^\star_8);$$

(c) for some $\eta \in \Omega_2(N)$ on the true branch, the exact stage-2 scan contains the true row $(K^\star_2, s^\star_3)$, and the stage-1 system

$$W_2(x) = E_1(x + s_1; K_1) + s_2$$

has a unique solution $(K^\star_1, s^\star_1, s^\star_2)$.

Then the implementation outputs the true 256-bit key and accepts it by the exact codebook test. No false full key can be accepted.

Proof. By assumption (a), the recovered outer pair is the true $K^\star_8$. Consider the recursive branch

$$K^\star_7, K^\star_6, \ldots, K^\star_3.$$

At stage t, assumption (b) says that

$$K^\star_t \in \mathcal{P}^{(N)}_t(K^\star_{t+1}, \ldots, K^\star_8).$$

Since the implementation eventually explores every candidate in that set, the true branch is never pruned. Therefore the recursive search reaches stage 2 on the true suffix.

Now fix the stage-2 IV η from assumption (c). On the true branch, the fixed-pair peeling equations (11)–(12) reconstruct the exact public current table

$$C^{(\eta)}_2(x) = R^{(\eta)}_2(x) + s^{(\eta)}_3.$$

Since the exact stage-2 scan includes the true row $(K^\star_2, s^\star_3)$, the reduction (15) holds for the true reduced table

$$W_2(x) = D_2(C^{(\eta)}_2(x) - s^\star_3; K^\star_2).$$

By assumption (c) and Theorem 1, the stage-1 closure returns exactly the unique true triple $(K^\star_1, s^\star_1, s^\star_2)$. Hence the reconstructed full key is the true implementation key. Finally, the acceptance test compares complete 2^{16}-entry tables, so no false full key can be accepted. □

8 Worked random-key recovery

8.1 Target key and successful IVs

As a representative difficult instance, consider the random key

$$K = \texttt{8FCD06ACB99FB6F77135F7ABCD27778AD08489BA2C6DE01CFC17FF0CE0F553C1}.$$

The successful public stagewise run used

- outer beam budget $N_{\text{out}} = 512$;
- beam size $B = 32$;
- search seed 1;
- inward budget $N = 64$ per target stage;
- successful stage-2 IV

$$\texttt{F71831A507C6103EBB51EC0AC07A0C60}.$$

8.2 Recovered values

The public recovery returned

$$\begin{aligned}
K_8 &= (\texttt{E0F5}, \texttt{53C1}), \\
K_7 &= (\texttt{FC17}, \texttt{FF0C}), & s_8 &= \texttt{675F}, \\
K_6 &= (\texttt{2C6D}, \texttt{E01C}), & s_7 &= \texttt{E454}, \\
K_5 &= (\texttt{D084}, \texttt{89BA}), & s_6 &= \texttt{328C}, \\
K_4 &= (\texttt{CD27}, \texttt{778A}), & s_5 &= \texttt{8D1B}, \\
K_3 &= (\texttt{7135}, \texttt{F7AB}), & s_4 &= \texttt{A2CF}, \\
K_2 &= (\texttt{B99F}, \texttt{B6F7}), & s_3 &= \texttt{6DEA}, \\
K_1 &= (\texttt{8FCD}, \texttt{06AC}), & s_2 &= \texttt{7F43}, & s_1 &= \texttt{27B3}.
\end{aligned}$$

The decisive public log line is

`full-key candidate verified against exact one-word oracle table`

from `stagewise_attack_hard1_default64.log`.

9 Monte Carlo estimate for full-program random-key success

9.1 Experiment design

This Monte Carlo evaluates the entire implemented attack, not a conditional inward probe. We sampled

$$m = 100$$

independent pseudorandom 256-bit keys by

$$K_j := \text{SHA256}(\texttt{``stagewise-mc-key-}j\texttt{''}), \qquad j = 0, \ldots, 99.$$

To isolate the dependence on the inward budget, we fixed the outer search profile to

$$N_{\text{out}} = 512, \qquad B = 32, \qquad \texttt{search-seed} = 1, \qquad \texttt{workers} = 16,$$

independently of the current binary defaults. We then evaluated the full public program

`separ_public_stagewise_attack.exe`

at inward budgets

$$N \in \{2, 4, 8, 16, 32, 64\}.$$

The budget-64 sample was the already completed 100-key run logged in

`monte_carlo_stagewise_100.log`.

For budgets $2, 4, 8, 16, 32$ we reran the same 100-key set and copied the existing budget-64 rows verbatim into the combined dump

`monte_carlo_stagewise_budget_sweep.log`.

Because of Proposition 1, success is monotone in N. Accordingly, within the budget sweep, once a key succeeded at a smaller budget, all larger budgets were recorded as monotone skips rather than rerun.

9.2 Observed success rates

Let

$$P_N := \Pr[\text{full public attack succeeds on a random key at inward budget } N]$$

under the fixed outer parameters

$$(N_{\text{out}}, B, \texttt{search-seed}) = (512, 32, 1).$$

For each budget we report the observed success rate together with the exact two-sided Clopper–Pearson 95% confidence interval. The measured success table is:

Budget N per stage	Total IV contexts	Chosen encryptions	Successes	95% exact CI
2	$512 + 32 + 6 \cdot 2 = 556$	36,438,016	9/100	[0.0420, 0.1640]
4	$512 + 32 + 6 \cdot 4 = 568$	37,224,448	45/100	[0.3503, 0.5527]
8	$512 + 32 + 6 \cdot 8 = 592$	38,797,312	89/100	[0.8117, 0.9438]
16	$512 + 32 + 6 \cdot 16 = 640$	41,943,040	100/100	[0.9638, 1]
32	$512 + 32 + 6 \cdot 32 = 736$	48,234,496	100/100	[0.9638, 1]
64	$512 + 32 + 6 \cdot 64 = 928$	60,817,408	100/100	[0.9638, 1]

Thus the random-key success curve rises sharply with the inward budget and reaches empirical saturation by $N = 16$ on the 100-key sample. At every tested budget $N \geq 16$, the observed success rate is 100%, with exact 95% lower confidence bound

$$\underline{P}_{95}(100/100) = 0.963783307\ldots$$

and corresponding one-sided statistical error bar

$$1 - 0.963783307 \approx 0.036216693,$$

that is, about 3.62 percentage points.

9.3 Runtime on the Monte Carlo sample

For the independently executed budget-64 sample of 100 keys, the measured wall-clock times on the user's machine were:

Statistic	Seconds
mean	52.520
median	52.346
min	30.358
max	89.096

Thus, under the explicit Monte Carlo profile $(N_{\text{out}}, B, N) = (512, 32, 64)$, the full public attack is a seconds-to-minutes computation rather than a brute-force search.

10 Complexity and bound

10.1 Oracle-query complexity

One full one-word codebook requires exactly

$$2^{16}$$

chosen encryptions. The implementation does not require chosen-ciphertext queries, so

$$Q_{\text{dec}} = 0.$$

For outer budget N_{out}, beam size B, and per-stage inward budget N, the successful direct path of the stagewise attack builds:

1. N_{out} outer-search codebooks;
2. B exact beam codebooks for the hybrid K_8 bootstrap;
3. N fresh-IV codebooks for each target stage $t = 7, 6, 5, 4, 3, 2$.

Hence the exact chosen-encryption count on a no-backtracking successful path is

$$Q_{\text{enc}}^{\text{direct}} = \big(N_{\text{out}} + B + 6N\big)2^{16}. \tag{18}$$

With the current binary defaults

$$N_{\text{out}} = 64, \qquad B = 8, \qquad N = 64,$$

this becomes

$$Q_{\text{enc}}^{\text{direct}} = (64 + 8 + 384)2^{16} = 456 \cdot 65536 = 29{,}884{,}416.$$

10.2 Exact branch-aware bound

The implementation is recursive and may, in principle, backtrack over exact candidate-pair sets before reaching the accepting branch. To state the exact implemented bound, let

$$M_t(N)$$

be the maximum, over all keys and all previously fixed suffixes $(K_{t+1}, \ldots, K_8)$ reachable by the program, of the cardinality of the exact stagewise candidate set

$$\mathcal{P}_t^{(N)}(K_{t+1}, \ldots, K_8).$$

Define recursively the maximum number of fresh-IV codebooks visited below stage t by

$$T_2(N) := N, \tag{19}$$

$$T_t(N) := N + M_t(N)\, T_{t-1}(N), \qquad t = 3, 4, 5, 6, 7. \tag{20}$$

Then the exact program-level chosen-encryption bound is

$$Q_{\mathrm{enc}} = \big(N_{\mathrm{out}} + B + T_7(N)\big)2^{16}, \qquad Q_{\mathrm{dec}} = 0. \tag{21}$$

Equation (21) is the correct exact bound for the implemented recursive search tree. Equation (18) is the realized cost when the true branch is reached without backtracking. In the 100-key Monte Carlo sweep, this no-backtracking behavior was observed on every successful key.

10.3 Exact local work

Once a codebook is built, all remaining work is local and exact. For each stage-t fresh IV, the local work consists of:

1. exact fixed-pair state recovery for each already-solved stage above t, using (10);
2. one exact stage-t low-byte scan;
3. one exact maximizing projected-cycle search over the 2^{20}-entry representative space;
4. exact order reconstruction by Hungarian assignment lifts;
5. exact candidate lifting and exact verifier minimization;
6. for $t = 2$, the exact stage-1 closure of Section 6 together with full-table verification.

10.4 Overall bound

For fixed budgets (N_{out}, B, N), the exact implemented query bound is (21). On the empirically dominant no-backtracking path, it collapses to the direct-path cost (18). For the default parameters of the current binary, the direct chosen-encryption count is

$$29{,}884{,}416.$$

Under the explicit Monte Carlo profile $(N_{\mathrm{out}}, B, N) = (512, 32, 64)$, the 100-key sample averaged 52.520 seconds per full key recovery, with median 52.346 seconds and worst case 89.096 seconds.

11 Implementation and runtime

The attack is implemented in the multithreaded C program

`separ_public_stagewise_attack.exe`.

The public end-to-end runner is

```
separ_public_stagewise_attack.exe --search-trials N --search-beam B
    --search-seed S --inward-trials M --workers W --oracle-key HEX64
```

with optional `--verbose` output. In the default mode the program shows only the current phase, progress, best IV, and recovered key segments in a compact in-place display.

Operationally, the public path:

- recovers K_8 only by the public hybrid score over the beam;
- recovers stages 7 through 2 only by the exact stagewise recursion of Section 5;
- recovers stage 1 only by the exact closure of Theorem 1;

- accepts a full key only after exact equality of the recovered codebook with the oracle codebook.

The default parameters of the current binary are

$$N_{\text{out}} = 64, \qquad B = 8, \qquad N = 64, \qquad W = 16.$$

The full-program Monte Carlo of Section 8 used the explicit larger outer-search profile

$$N_{\text{out}} = 512, \qquad B = 32, \qquad \texttt{search-seed} = 1, \qquad W = 16$$

in order to measure the dependence of success probability on the inward budget N alone.

12 Conclusion

The `main.c` implementation of SEPAR is broken end-to-end in the chosen-IV, chosen-plaintext, chosen-ciphertext, reset model. The decisive structural fact is that every stage is exposed through an exact translated 2^{16}-point permutation, and once a suffix stage key is fixed the corresponding IV-local state word can itself be recovered exactly and peeled away on fresh IVs. This yields the stagewise recursion of Section 5 and removes the dependence on one exceptionally good inward IV.

The attack is therefore not a heuristic key-ranking experiment. Conditional on the sampled IV sets containing the true exact candidates, the recovery is deterministic and exact; false full keys are excluded by complete one-word table equality. On a 100-key Monte Carlo sample of the full program, the explicit profile $(N_{\text{out}}, B, \texttt{search-seed}) = (512, 32, 1)$ achieved success rates $0.09, 0.45, 0.89, 1, 1, 1$ at inward budgets $N = 2, 4, 8, 16, 32, 64$, respectively, with exact 95% lower confidence bound 0.9638 once $N \geq 16$.